

安卓恶意应用的大规模时序测量：持久性、迁移性及经验教训

训

云深
Norton 研究小组

皮埃尔-antoine · 维尔维
Norton 研究小组

吉安卢卡 · 斯特林吉尼
波士顿大学

摘要

我们通过利用 2019 年至 2020 年间一款流行移动安全产品 1170 万用户收集的 880 万次每日设备检测数据，研究了安卓设备上潜在有害应用（PHAs）的时间动态。我们发现，当前的安卓安全模型限制了安全产品只能作为普通应用运行，并阻止它们自动移除恶意应用，这为攻击者提供了显著的机会。这类应用会向用户警告新发现的威胁，但用户不会及时采取行动，导致 PHAs 在被检测后平均在设备上持续存在 24 天。我们还发现，虽然应用市场在 PHAs 被公开后将其移除，但 PHAs 从被识别到被移除之间存在显著延迟：PHAs 在 Google Play 上平均持续存在 77 天，在第三方应用市场则持续 34 天。最后，我们发现 PHAs 在被原始市场移除后会迁移到其他市场。本文提供了前所未有的安卓 PHA 景观视角，表明当前针对安卓 PHA 的防御措施并不像普遍认为的那样有效，并确定了安全社区应追求的多个研究方向，从组织更有效的 PHA 打击行动到为移动安全产品设计更好的警报机制。

获取 Android，谷歌创造了“潜在有害应用”（PHAs）这一术语。

已发表大量研究测量 PHA 对 Android 的威胁。先前研究主要依赖于爬取应用市场以检索恶意应用[1, 23, 31, 34, 40]。替代方法包括从公共仓库下载固件以研究预装 Android 应用[11]，以及建立依赖第三方提交他们怀疑的恶意应用的公共分析基础设施[21]。这些方法随后通过静态或动态分析来分析收集到的应用。虽然这些方法有助于揭示恶意 Android 应用的功能，但它们无法了解感染设备的数量以及用户如何与 PHA 交互。另一种方法依赖于用户安装一个能够监控设备网络流量的应用，寻找安全和隐私敏感信息[26]。该解决方案解决了上述问题，但招募大量具有代表性的用户群体具有挑战性；事实上，先前研究依赖于 11k 用户来执行他们的测量[26]。研究人员采用的第三种方法是监控移动互联网服务提供商的网络流量，并根据黑名单信息识别恶意连接[18]。这种方法能够实时查看大量设备上的恶意活动，但仅限于监控与已知恶意主机的连接。此外，这种方法受限于广泛使用加密，例如，它允许观察设备何时连接到应用商店，但不必检查用户正在安装的具体 PHA。

在这篇论文中，我们展示了首个大规模研究，旨在理解安卓设备上 PHA 安装的时间动态。我们从安装了流行移动安全产品的用户那里收集了匿名的 PHA 安装信息，这些用户选择参与数据收集。在 2019 年至 2020 年间，我们观察到超过 880 万 PHA 安装在超过 1170 万设备上。

1 引言

多年来已观察到数百万个恶意安卓应用[1]，它们执行各种恶意活动，从发送高级短信[23]到显示烦人广告[31]再到实施跟踪[5]。安卓上的恶意应用通常以重打包应用的形式出现，即一个有用的安卓应用被修改以包含隐藏的恶意功能，以诱使用户安装它[20, 31, 39]。为了涵盖各种恶意应用，这些应用会...

数据集数据计数
移动 PHA 检测日志 总记录数 3.2B (2019 年 1 月 - 2020 年 2 月) 天数 416
国家和地区 201 设备 11.7M 独特应用名称 2.3M 独特应用 SHA2 8.8M VT 总报告数 8.8M
PHA SHA2s (检测 ≥ 4) 7M 单例 SHA2s (不含家族) 1.3 M PHA SHA2s 含家族 5.7 M PHA 家族 3.2K

表 1: 数据集摘要。

在全球范围内。这些数据使我们能够开发出多种指标，并回答以下关键研究问题：

设备感染 PHAs 多久？移动

在 Android 上运行的安全产品通常以普通用户身份运行，没有 root 权限，因此它们在检测到恶意程序后能采取的行动受到限制。通常情况下，它们只是发出警报，告知用户存在该问题，并依赖用户自行卸载恶意应用。我们的研究表明，用户对这些警报反应迟缓，一旦检测到 PHA，它们会在设备上持续存在大约 20 天。

PHAs 在应用商店中能存活多久？通过观察数百万部移动设备从应用商店安装恶意应用，我们可以估算某个 PHA 何时被从商店中移除。我们发现，平均而言，PHAs 在 Google Play 上持续存在 77 天，而在其他应用商店中平均持续 34 天。

PHAs 被移除后是否迁移到其他应用市场？

我们观察到有 3,553 个 PHAs 展现出跨市场迁移的现象。然而，这些 PHAs 在这些市场上的平均寿命与平均持久时间相比要短。

如果从备份/克隆服务迁移，PHAs 在设备上的持久时间会更长吗？Android 设备允许用户备份他们的应用，并在用户更换新手机时自动在新设备上安装。我们发现这些 PHAs 在这些设备上的平均持久时间更长。例如，我们发现了 14K 个 PHAs 通过使用三星智能切换移动应用 (com.sec.android.easyMover) 迁移到 35.5K 部新三星设备。这些应用在新设备上的平均持久时间为 93 天。

对安卓恶意软件研究的影响。我们的研究对计算机安全研究界具有若干影响。我们表明，恶意应用可以在应用市场上存活很长时间，并且安卓安全模型严重限制了移动安全产品在检测到恶意应用时能做什么，允许 PHAs 在受害者设备上持续存在多日。此外，我们的结果表明，当前移动安全程序采用的警告系统并不能有效说服用户及时卸载 PHAs。这可能是由于可用性问题，如警报疲劳[28]，并呼吁在此领域进行更多研究。我们还表明，恶意应用开发

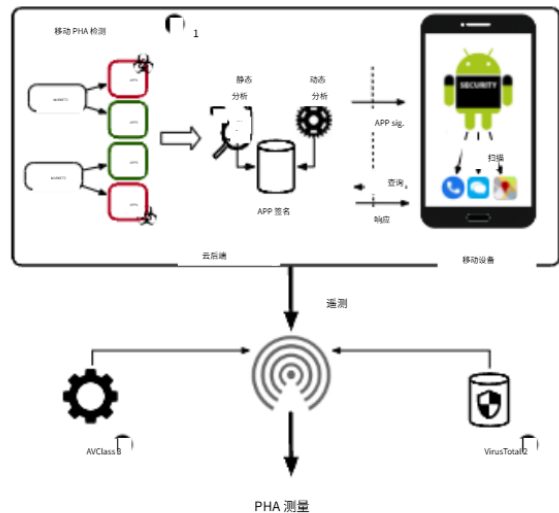


图 1: 数据收集架构。

在它们被移除后，经常将它们的 PHAs 转移到其他市场。这表明一个有效的缓解策略应包括多个市场之间的合作。

2 数据

本节总结了我们的数据收集方法（见图 1）以及本研究使用的数据集（见表 1）。

移动 PHA 声誉数据 ∂ 。在本文中，我们使用 mo-

由 NortonLifeLock 的移动安全产品收集的真实世界安卓设备上的恶意软件声誉数据，覆盖全球 200 多个国家和地区。与 Kotzias 等人[16]讨论的设备地理位置分布类似，我们研究中 25% 的设备来自美国，28% 的设备来自欧洲国家，31% 的设备来自亚太地区（尽管这种分布偏向日本和印度）。

诺顿生命锁拥有弹性基础设施，可大规模收集并执行对来自多个市场和合作伙伴的移动应用的静态（例如，流程和上下文敏感的污点分析、细粒度权限分析）和动态分析（例如，APK 模糊测试、UI 自动化、在沙盒环境中检查网络跟踪、行为等）。在此过程中，恶意活动及其触发条件（如反射、动态代码加载、本地代码执行、请求与其宣传描述无关的权限等）被分析和指纹识别。诺顿生命锁采用最先进的商业产品来应对模拟器/运动规避、混淆代码/库以及其他规避技术等挑战，并触发应用中的关键执行路径。结果随后被纳入诺顿生命锁的检测引擎，并在其移动安全产品中部署以扫描和识别可疑

移动终端上的应用程序。NortonLifeLock 还从已知恶意软件的静态和动态分析结果中构建机器学习模型，并将这些模型应用于检查未知或低流行度应用程序。此外，应用程序会定期由分析基础设施重新检查。

在运行时，移动安全产品会定期扫描设备上新安装的应用程序，并在用户请求时执行全设备扫描。当设备有网络访问时，安全引擎会查询云后端以获取设备上安装的应用程序的判定结果。查询包含某些元数据，包括时间戳、应用哈希值、包名、证书信息等。后端的响应包含设备上应用程序的信誉分数以及其他专有数据，以指导后续操作。当没有网络访问时，安全引擎会利用本地存储的签名来扫描和识别移动端的可疑应用程序。一旦网络访问恢复，相应的扫描元数据将被发送回去。

从后端遥测数据湖中，我们提取以下信息：匿名化设备标识符、设备国家代码、检测时间戳、应用 SHA2、应用包名和安装程序包名。通过这种方式，我们能够确定 PHA 检测的时间、安装的设备以及安装它的包。我们在 2019 年 1 月 1 日至 2020 年 2 月 20 日期间收集了 416 天的检测数据。平均每天收集 8M 原始事件（即总共 3.2B 事件）。请注意，为了进行时间测量，我们仅选择在同一设备上至少观察到两次的应用（按 SHA2）。这样，我们可以可靠地计算它们在设备上和在市场上的生命周期（见第 3.2 节）。总共，我们的数据集涵盖了 2.3M 个唯一包名和 8.8M 个唯一 SHA2，来自 11.7M 个设备。我们在第 9 节提供了关于我们数据集可能产生的偏差的详细讨论。

VirusTotal Σ. 注意到不同安全公司在标记 PHAs（尤其是广告软件）时有不同的策略。也就是说，由参与这项研究的 NortonLifeLock 标记的 PHA 可能不会得到其他安全公司的共识。为了最小化数据集可能产生的误报和偏差，我们在 VirusTotal 上查询与我们数据集中的 PHAs 对应的 8.8M 个 SHA2。如果 VirusTotal 在这篇论文中返回至少四个检测结果，我们就认为该应用是 PHA。这符合最近由恶意软件研究社区提出的最佳实践 [16, 41]。我们建议读者参考 Kotzias 等人 [16] 和 Zhu 等人 [41] 对 VirusTotal 报告的不同检测阈值影响的深入分析。总共，我们识别出 7M 个独特的恶意 SHA2，以及 3.5K 个 PHA 家族。

AVclass Π. 在我们的研究中，可靠的 PHA 标记是保证恶意软件家族归属质量的一个必要条件。为此，我们使用 AVclass [29] 来提取家族

来自杀毒软件标签的信息。该工具从给定 PHA 的 VirusTotal 报告中选择多数投票对应的最顶级家族，有效去除标签中的噪声。总共，观察到的 PHA 属于 3.2K 个家族。并非所有 PHA 都具有同等危害性。虽然有些应用显然是恶意的（即包括勒索软件、木马、间谍软件等的移动恶意软件），但其他一些应用仅仅是给用户带来烦恼（例如广告软件）。谷歌将这些应用归类为移动不良软件（MUwS），称为“并非严格意义上的恶意软件，但对软件生态系统有害的应用” [12]。为了研究恶意软件和 MUwS 行为上的差异，我们使用 AVclass 提供的功能将样本分类为移动不良软件（MUwS）或移动恶意软件（见第 5 节）。请注意，E [13] 也挖掘了杀毒软件标签，并分析了不同供应商给出的所有标签之间的关联，将常见的样本统一归类为家族组。由于 AVclass 在家族归属方面的标签准确性相当，且所需内存较低，我们在本文中选择 AVclass。

数据蒸馏和测量数据选择。

研究 PHAs 的来源，特别是它们安装自哪些市场，我们需要收集检测到的 PHAs 的安装程序包名称信息。移动安全产品使用 Android API 在检测事件触发时记录 PHAs 的安装程序包名称。然而，由于 Android 设备制造商的众所周知碎片化以及我们测量基础设施的限制（例如，我们无法通过 Android API 识别安装程序包的证书），很难准确提取和归因所有检测到的 PHAs 的安装程序包。例如，如果在观察期开始前应用程序已在设备上安装，我们的方法将无法将其归因于安装它的应用程序。类似地，如果安装了现有 PHA 的更新版本，这将被识别为通过更新组件安装，而不是通过市场（例如，com.google.android.packageinstaller）。为了缓解这个问题，我们首先识别出 1170 万部中有 370 万部至少安装了一个 PHAs 的设备。然后通过选择 2 来精炼上述数据集。46M 设备中，我们可以高度自信地将它们的设备端 PHAs 归因于相应的安装包。总共，我们从 2.46M 设备中识别出 197K 个 PHA，这些 PHA 用于第 6 节和第 7 节来研究 PHA、设备和市场之间的动态关系。这些 PHA 占我们数据集在观察期间记录的所有安装的 22%。我们在第 9 节对这种方法进行了详细的讨论，并指出了其局限性。

伦理与数据隐私。本文使用的数据涉及隐私。NortonLifeLock 为终端用户提供明确选择加入其数据共享计划的可能性，以帮助提高安全产品的检测能力。当应用程序运行时，在设置过程中会显示此对话框。

符号说明	
$p \in P$	一个 PHA
$d \in D$	一个设备
$m \in M$	一个市场
$f \in F$	一个 PHA 家族
x	x 在 y 上/中, 例如, p denotes 在设备 d 上检测到的 PHA。
(F)	首次出现时间戳, 例如, p denotes 设备 d 上 PHA p 的首次出现时间戳。
(L)	最后一次出现时间戳, 例如, p denotes 市场 m 中 PHA p 的最后一次出现时间戳。
δ	x 在 y 上的寿命, 例如, δ denotes 设备 d 上 PHA p 的寿命。

表 2: 本文中使用的符号摘要。我们用小写字母表示项目, 用粗体大写字母表示集合。

这是首次, 它向最终用户说明了遥测数据收集的目的, 以及 NortonLifeLock 的全球隐私政策如何保障数据安全。例如, 许可协议规定遥测数据“用于通过提醒您潜在的恶意应用程序、恶意软件和链接来交付产品”, 以及“用于了解产品使用情况, 以进一步开发和改进产品性能以及遥测数据”。由于本文的分析使社区能够更好地理解 Android PHA 生态系统并指导缓解技术, 这属于用户同意的数据主要用途。遥测数据的收集、存储和处理受到 NortonLifeLock 严格隐私政策的保护。为保护用户及其设备的匿名性, 客户端标识符已被匿名化, 并且无法将收集到的数据与用户和原始移动设备关联起来。此外, NortonLifeLock 不会追踪设备或分析用户行为, 也没有检查网络数据的能力。在我们的测量研究中, 匿名设备标识符仅用于计算基于设备的流行率。因此, 我们并未使用任何个人身份信息, 对用户的风险极小。

3 方法

在本节中, 我们首先介绍 PHAs、安装程序包、设备和市场之间的整体关系。然后, 我们结合实例描述了我们的整体测量设计理念和方法。

3.1 关系

为了方便读者, 我们将此处及以下各节中引入的符号总结在表 2 中。我们详细描述数据集中观察到的关系, 为本文其余部分的测量提供基础。图 2 展示了一个示例, 说明 PHAs P 、安装包 Ψ 、设备 D 和市场 M 之间的复杂动态关系, 并附带时间线。每

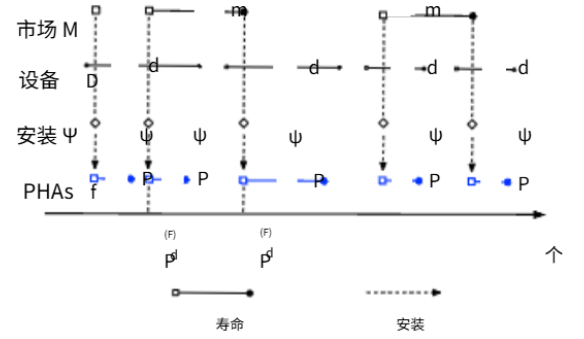


图 2: 根据我们的数据集观察到的 PHAs、安装商、设备和市场之间关系的抽象模型。

设备 d 可以安装多个 PHAs (例如, 图 2 中 d 有两个 PHAs p 和 p)。一个 PHA p 可以存在于多个设备中 (例如, p 安装在了所有四个设备中)。此外, 多个 PHAs 可以属于一个 PHA 家族。例如, 如图 2 所示, P 包括 p 和 p , 在所有四个设备中。此外, Android API 允许移动安全产品获取安装了 PHA 的应用的包名 (即 ψ)。这使我们能够识别 PHA 来自哪个市场, 如果 ψ 的包名与市场的名称匹配。例如, 设备 d 上的 p 是由来自市场 m 的包 ψ 在时间戳 p 安装的。

(见图 2)。将同一 PHA 引脚的所有安装事件聚合到所有设备 D 中, 我们可以估计 δ 在 市场 m 的寿命为 $[p_j, p]$ (见图 2)。

3.2 设计理念

由于我们并非应用市场的所有者, 测量 PHAs (例如, PHAs 被移除的速度) 是一项具有挑战性的任务。一种解决方案是爬取已知的应用市场并每日跟踪所有应用 [35]。然而, 爬取结果无法与应用的设备安装数据进行关联, 因为并非所有市场都提供精确的设备安装信息。在本研究中, 我们采用一种自外而内的设计理念来执行我们的市场存在度测量。也就是说, 我们将移动设备视为传感器, 并将它们的 PHA 安装事件视为 PHA 存在与否的探测结果。然后, 我们利用应用安装包的信息来识别已安装 PHAs 的来源市场 (参见上文关于关系的部分)。通过将此信息与设备上的检测时间戳进行关联, 我们可以以非侵入性、自外而内的方式计算 PHA 的市场持续性和流行度。类似地, 我们也可以使用检测时间戳来计算设备上的 PHA 持续性。在本研究中, 我们沿三个轴使用不同的指标来研究 PHA 生态系统: 设备上的持续性、市场持续性和 PHA 跨市场迁移。在这一节中, 我们定义了后来将用于衡量这三个方面的指标。

3.2.1 设备上 PHA 的持久性测量

移动安全产品在后台周期性运行，如果检测到 PHAs，会向后端发送遥测数据。如果用户收到警报后设备上未移除 PHA，移动安全产品会记录在不同时间戳下的重复检测，直到设备上移除 PHA。基于这一系列检测事件，我们可以得知 PHA 在设备 d 上的首次和最后一次出现的时间戳，从而能够估计 PHA 在设备 d 上的存活时间（即 δ ）。基于这一观察，我们使用公式 1 来测量 PHA 家族 f 在设备 d 上的持久期。

$$\text{persistence}(f, d) = \frac{(\delta)}{|\mathcal{P}|} \quad (1)$$

也就是说，我们计算属于家族 f 的所有 PHA 在设备 d 上的平均寿命。例如，在图 2 中，家族 f 在设备 d 上有两个 PHA（p 和 p），因此 $\text{persistence}(f, d) = (\delta + \delta)/2$ 。然后我们使用公式 2 来测量所有设备 D 上每个 PHA 家族 f 的平均持续期。

$$\text{persistence}(f, D) = \frac{\sum_{d \in D} \text{persistence}(f, d)}{|D|} \quad (2)$$

例如，在图 2 中的所有四个设备中，家族 f 都有存在。根据公式 2，我们可以计算 $\text{persistence}(f, D)$ 为 $[\text{persistence}(f, d) + \text{persistence}(f, d) + \text{persistence}(f, d) + \text{persistence}(f, d)]/4$ 。

3.2.2 PHA 市场内持久性测量

给定单个设备 d，当移动安全产品在移动设备上检测到 PHA 时，它还会记录该 PHA 的安装包名称。通过与市场官方包名称进行关联，我们可以识别出某个特定市场 m 在某个特定时间戳是否安装了 PHA。例如，如果我们观察到某个 PHA 的安装包名称是 com.android.vending，我们可以确定这个 PHA 来自 Google Play 商店。请注意，恶意应用可以冒充 Android 设备上的合法应用（例如，com.android.vending 可能不是合法的 Google Play 应用）。为了避免错误归因，我们检查同一设备的检测遥测数据，并验证是否有任何检测记录与已知市场的相同包名称匹配。通过这样做，我们能够验证本研究中市场应用的合法性。我们在第 9 节提供了对该方法局限性的详细讨论。请注意，设备 d 上 PHA 的首次出现时间戳可以可靠地证明 PHA 在安装时存在于某个市场。通过在所有设备 D 上聚合 PHA 的首次检测事件，我们可以使用公式 3 来表示 PHA 的市场出现情况。

$$\Omega = \{p_i\}, \forall d \in D, p \in \mathcal{P}(3)$$

本质上， Ω 表示一系列时间戳，其中 p 在所有设备 D 上首次被看到。以图 2 中的关系为例，我们有两个 PHA p 分别在 d 和 d 上被检测到，这些检测来自市场 m。相应地，我们有 $\Omega = \{p_i\}$ 。

根据上述观察，我们使用公式 4 来测量市场 m 中一个 PHA 引脚的持久周期。

$$\text{persistence}(p, m) = \max(\Omega) - \min(\Omega) \quad (4)$$

可以直观地观察到 $\text{persistence}(p, m) = p -$

遵循公式 3 和公式 4。请注意，我们依赖设备上的检测来测量一个 PHA 在市场上的持续存在。有可能一个 PHA 仍然存在于市场上，但我们的数据集没有反映其存在。因此，我们测量了 PHA 在市场上持续存在的下限。最后，我们使用公式 5 来测量 PHA 家族 f 在市场 m 中的持续期。

$$\text{persistence}(f, m) = \frac{\sum_{p \in \mathcal{P}} \text{persistence}(p, m)}{|\mathcal{P}|} \quad (5)$$

3.2.3 PHA 跨市场迁移测量

回想移动安全产品记录了 PHA p 在设备 d 上由安装包 ψ 在时间戳 t 安装。通过聚合关于特定 PHA p 的遥测数据，并将其安装包名称映射到所有设备 D 中的各个市场 M，我们可以跟踪 PHA p 在所有市场 M 中的出现情况。以图 2 中的 PHA p 为例，它在两个市场 m 和 m 中的四个设备（d，d，d 和 d）中被检测到。根据公式 3，p 在 m 和 m 中的生命周期分别是 $\delta = [\min(\Omega), \max(\Omega)]$ 和 $\delta = [\min(\Omega), \max(\Omega)]$ 。如图 2 所示，我们观察到 $\max(\Omega)$ 小于 $\min(\Omega)$ ，因此我们定义 PHA p 从 m 迁移到 m。根据这一观察，我们使用公式 6 来表示 PHA p 在市场 M 中的出现情况。

$$\text{appearance}(p, M) = \{\delta\}, \forall m \in M \quad (6)$$

请注意，每个 δ 是一个区间（即 $[i \min(\Omega), \max(\Omega)]$ ）。然后，我们按 $\min(\delta)$ 对 $\text{appearance}(p, M)$ 进行排序，然后从 $\text{appearance}(p, M)$ 中依次识别不重叠的区间，以测量 PHA 在市场 M 中的跨市场迁移。

3.3 右删失数据

当某些个体的生存时间信息不完整时会发生删失。回想一下，我们的观察期是在 2019 年 1 月 1 日到 2020 年 2

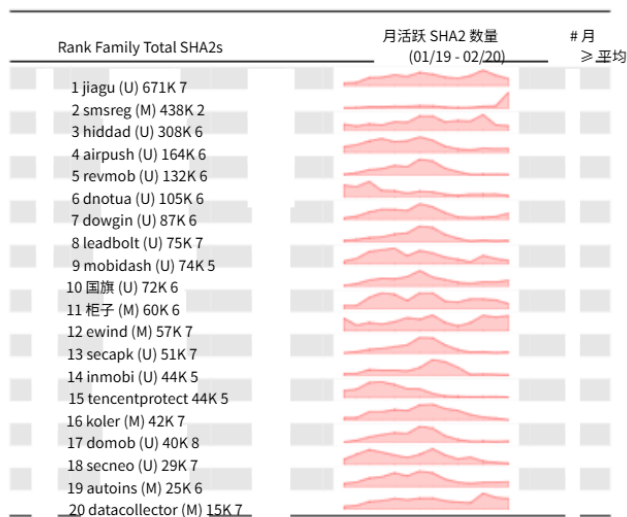


表 3：前 20 个 PHA 家族的时间模式总结。这些家族按 SHA2 总数排序。M 表示恶意软件，U 表示 MUwS/广

2020 年，存在一些 PHAs 在我们研究于 2020 年 2 月 20 日结束前已从市场上移除，因此我们无法观察到它们。此类 PHA 数据在生存分析中定义为右删失数据 [14]。在我们的研究中，我们假设删失与发生感兴趣事件（即 PHA 移除）的可能性独立或不相关。因此，我们保留这些右删失数据以避免估计偏差。更多细节可参见第 6 节。

4 PHA 家族的时间特征

恶徒们要么将多个使用相同恶意代码的应用重新打包，要么修改代码以避免被安全措施检测到 [23, 39]。这些相关的恶意应用通常被称为家族。在本节中，我们研究 PHA 家族的时间模式（例如，我们是否总是看到相同的顶级家族，或者由于移动安全公司检测力度的增加而逐渐减少其运营？）。如第 2 节所述，我们使用 AVclass [29] 将应用分组为家族。表 3 显示了前 20 个 PHA 家族的时间流行度，并总结了我们的发现。这前 20 个 PHA 家族被检测到在 201 万部设备上，大约是上述 370 万部设备的 67%。回想一下，我们选择至少在相同设备上观察到两次的 PHA（按 SHA2）来进行测量（见第 2 节），因此我们的数据集特意设计用于测量时间行为。我们的结果与最近的一项 Android PHA 设备流行度研究 [16] 一致：表 3 中的 20 个 PHA 家族中有 15 个也出现在 [16] 发现的顶级 PHA 家族中。如表 3 所示，20 个顶级 PHA 家族中有 16 个成功在至少 6 个月内超过其月活跃 SHA2 数量的平均值。此外，我们看到大多数

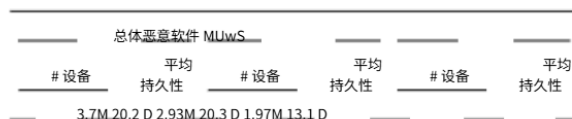


表 4：设备上的整体 PHA 持久性。

PHA 系列表现出钟形的时间模式。这表明这些 PHA 系列可能最终会减少其运营，因为移动安全公司和市场在检测方面加大了力度。我们将在第 6 节（例如，如何快速下架可以破坏 PHA 运营）进一步分析这一发现，以及在第 7 节中，恶意行为者如何将他们的 PHA 转移到替代市场。此外，我们注意到 smsreg 和 smspay 在我们的观察期结束时（即 2020 年 1 月和 2 月）显示出上升趋势。鉴于最近关于基于 SMS 的双因素认证限制的讨论，我们的发现表明此类漏洞在野外仍然存在，并引起了网络犯罪分子的注意。

5 设备上的 PHA 持久化

如我们所解释，移动安全产品受限于 Android 安全模型，一旦检测到 PHA 就无法删除它们。相反，它们通常会告知用户新发现的威胁，要求用户手动删除应用。这就留下了用户如何及时从设备中删除已识别的 PHA 的问题。在本节中，我们首先研究设备上的 PHA 持久化，了解 PHA 一旦安装后能在设备上持续多长时间。然后我们研究 PHA 设备上持久化的后果，例如，这是否会导致设备上安装更多的 PHA。

5.1 不同类型 PHA 的设备上持久化

如前所述，并非所有安卓 PHAs 都具有同等危害性，有些只是对用户造成烦扰（MUwS）。因此，当安全产品告知用户已安装恶意软件与移动非期望软件（MUwS）时，用户的反应可能会有所不同，可能不会卸载后者。为了更好地理解这一点，我们使用 AVClass [29] 在 PHAs 中将移动恶意软件与 MUwS 区分开来。然后我们遵循第 3.2 节中概述的方法，测量 PHAs 在设备上的整体持久性。我们的研究结果总结在表 4 中。我们发现，PHAs 一旦安装，会在设备上持续大约 20 天。平均而言，移动恶意软件的持久时间比 MUwS 更长（分别为 20.3 天和 13.1 天）。令人惊讶的是，终端用户在检测到 PHAs 后并未及时移除。PHAs 在设备上的持续存在为

2<https://krebsonsecurity.com/2018/08/reddit-breach-highlights-limits-ofsms-based-authentication/>

家族	平均持久性	最大持久性	# 设备 (≤ 平均值)	# 设备 (> 平均值)
jiagu	4.77 D	414.0 D	1132K	303K smsreg 2.37 D 413.63 D
471K 34K hiddad	5.54 D	415.08 D	611K 85K airpush	20.9 D 413.5 D 183K 35K revmob 3.54 D 413.82 D 354K 13K dnotua 2.93 D
414.36 D 261K 12K dowgin	7.24 H	412.11 D	239K 1K leadbolt	5.74 D 413.45 D 243K 12K mobidash 1.8 D 415.09 D 294K 10K
kuguo	6.18 H	408.89 D	155K 1K locker	4.13 H 413.13 D 195K 322 ewind 8.72 D 414.11 D 118K 22K secapk 12.98 H 412.03 D 215K
1K inmobi	10.66 D	413.77 D	213K 23K tencentprotect	5.27 D 413.58 D 173K 23K koler 0.49 H 360.01 D 160K 1K domob 1.87 D
409.91 D 174K 2K secneo	3.32 D	413.15 D	157K 11K autoins	22.45 D 414.0 D 174K 43K datacollector 15.47 D 413.59 D 176K 54K

表 5：设备上 PHA 家族前 20 名的持久性总结。D 表示天数，H 表示小时。

攻击者可以在其中对受害者及其设备造成损害的时间（例如，显示侵入性全屏广告、收集私人信息、在用户同意的情况下安装其他恶意应用程序）。

5.2 PHA 家族设备上持久性

我们随后遵循第 3.2 节中概述的方法，了解按设备流行率排名的前 20 个最大的 PHA 家族的设备持久性。我们的研究结果总结在表 5 中。有趣的是，在 20 个顶级 PHA 家族中，有 15 个可以在设备上持续数天，而只有 5 个 PHA 家族在不到两天内被用户移除。例如，ewind 是一个木马家族，它在 118K 设备上平均持续了 8.72 天，甚至在 22K 设备上持续时间更长。这一点很有趣，因为它表明用户在被移动安全产品警告后选择不卸载恶意应用。鉴于这一点，有趣的是观察到，用户在检测到后迅速移除了 locker/koler。我们只能推测其背后的原因可能是用户体验的下降（即 locker 屏幕锁定，koler 伪造 FBI 警告）。我们希望我们的研究结果能够使移动安全公司设计出有效的通知系统，在检测到 PHA 时促使最终用户将其移除，例如考虑到警报疲劳[28]。

5.3 PHA 多实例持久性

用户未及时删除检测到的 PHAs 这一事实，为攻击者创造了一个机会窗口（如表 4 所示），使攻击者能够更新已安装的 PHAs、在未经用户同意的情况下安装额外的恶意应用，或通过全屏广告诱使用户安装应用。我们将这种现象称为多实例持久化。图 3 显示了

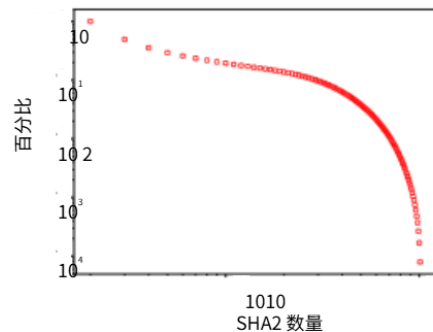
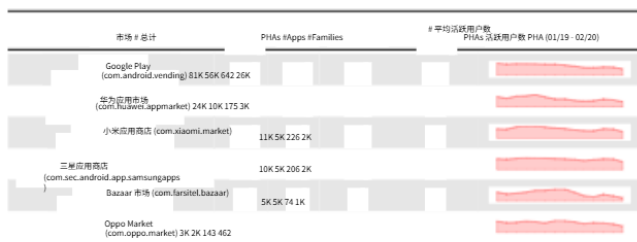


图 3：我们数据集中设备上 SHA2 数量的 CCDF（对数刻度）。

家庭 # 设备	# 多实例持久化设备	比率
jiagu 1.33M 849K 0.64 smsreg 499K 237K 0.48		
hiddad 670K 208K 0.31 airpush 214K 43K 0.2 revmob 367K 186K 0.51 dnotua 272K 82K 0.3 dowgin 241K 80K 0.33 leadbolt 255K 85K 0.33 mobidash 304K 112K 0.37 kuguo 155K 34K 0.22 locker 196K 53K 0.27 ewind 139K 17K 0.13 secapk 216K 75K 0.35 inmobi 235K 62K 0.27 tencentprotect 194K 42K 0.22 koler 161K 45K 0.28 domob 176K 45K 0.26 secneo 167K 34K 0.21 autoins 209K 33K 0.16 datacollector 211K 45K 0.21		

表 6：每个 PHA 家族的多实例持久化摘要。

我们数据集中设备上安装的唯一 PHAs 数量的补充累积分布函数（CCDF）。在观测期间，安装了 PHAs 的大部分设备安装了多个。例如，810K 部移动设备（占至少安装了一个 PHAs 的 3.7M 设备的 21.6%）安装了超过 7 个 PHAs。在本节中，我们研究了在设备上存在 PHAs 在多大程度上促进了其他恶意组件的安装。我们的研究结果如表 6 所示。前 20 个 PHA 家族中有 18 个在至少 20% 的受感染移动设备上表现出多实例持久性。例如，在 14 个月的观测期间，从 smsreg 家族安装了 PHAs 的 499K 部移动设备中有 237K 部至少还有两个来自同一家族的其他 PHAs。即使对于 locker 和 koler，尽管最终用户迅速采取行动（见表 5），我们观察到 53K 部（占 locker 感染设备的 27%）和 45K 部（占 koler 感染设备的 28%）表现出多实例持久性。请注意，我们的数据不允许我们推断 PHAs 安装的因果关系。我们的结果仅



我们的结果仅限于表 7：前 6 个 Android 市场的活跃 PHA 摘要。

证明来自同一家族且安装在相同设备上的两个 PHAs 高度相关。

6 PHA 市场内持久性

在本节中，我们首先利用我们能够可靠地确定其市场来源的 197K 个应用数据集（见第 2 节），量化了六个应用市场中的活跃 PHAs。如前所述，这个数字低于安装的 PHAs 总数，原因有多个（例如，在我们研究开始前就已经安装在设备上的 PHAs，以及不属于新安装应用而是更新版本的 SHA2），但它仍然涵盖了安装了任何 PHA 的 66% 的设备以及我们数据集中 22% 的 PHA 安装。然后，我们研究市场对 PHAs 存在反应的方式（例如，市场暂停或删除了多少 PHAs 等）。最后，我们研究 PHA 的市场内持久性（即 PHAs 发布后能在不同市场中持续多久）以及 PHA 的市场内演化（即 PHAs 如何演化以规避应用审核系统）。

6.1 PHA 市场内普遍性

移动安全产品记录了在移动设备上观察到的 PHAs 的安装包名称（见第 2 节）。这使我们能够追踪已安装 PHAs 的来源市场。我们首先测量 PHAs 的市场普及率，作为理解第 6.3 节中 PHAs 市场持续性的基础。我们调查了六个流行的安卓市场（即 Google Play、华为市场、三星市场、小米市场、Bazaar 市场和 Oppo 市场）中的活跃 PHAs。我们的结果总结在表 7 中。Google Play 是承载最多 PHAs 的安卓市场：642 个 PHA 家族的 81K 个唯一 SHA2，平均每月拥有最大的活跃 PHA 人口（即每月 26K）。这很合理，因为 Google Play 是最大的安卓市场，拥有大约 287 万个应用，因此成为 PHA 制作者的法定目标。总体而言，所有市场都表现出 PHAs 的持续月度存在，如表 7 中的时间模式所示。根据这一发现，我们将

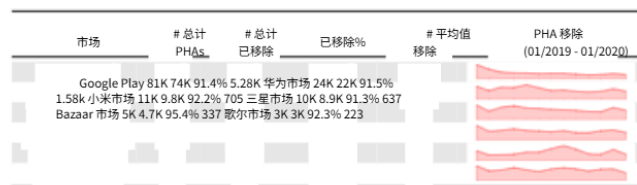


表 8：前 6 大安卓市场的 PHA 移除情况总结。

讨论这些市场如何处理 PHAs 将在下一节进行。

6.2 市场针对 PHA 的行动

当应用程序提交到 Android 市场时，它们通常会自动分析是否存在恶意活动。如果未被检测到，PHA 将在市场上发布，但后来可能会被暂停或删除，例如在 PHA 被用户或安全研究人员报告为恶意之后。谷歌在其年度 Android 安全和隐私报告中披露了 PHA 安装的百分比 [12]。然而，目前仍不清楚所有市场暂停或删除了多少 PHA。正如我们在上一节所示，所有市场都持续每月存在 PHA。例如，Google Play 在 14 个月内拥有 26K 个每月活跃的 PHA 和 81K 个总 PHA。这意味着市场确实会删除 PHA，但不是立即删除。为了更好地理解这一现象，我们遵循第 3 节的方法，测量前 6 个 Android 市场暂停或删除的 PHA 数量。请注意，我们定义一个 PHA 被移除/暂停，如果在它最后一次出现（即 $\max(\Omega)$ ）之后，我们不再观察到相同的 SHA2（即在我们的观测期间内）。这样，我们排除了所有在 2020 年 2 月出现的 SHA2，以最大程度地减少误报，并在第 9 节讨论这种策略的局限性。我们的研究结果总结在表 8 中。总体而言，每个 Android 应用商店至少移除了在我们观测期间发布在其中的 91.3% 的 PHA。例如，Google Play 移除了 74k 个 PHA（占 81k 个 PHA 的 91.4%），而 Bazaar 市场表现最佳，从其应用商店移除了 4.7k 个 PHA（占 5k 个 PHA 的 95.4%）。与之前的工作[35]不同，我们发现像华为、小米和 OPPO 这样的中国市场也在移除它们发布的大部分 PHA（分别为 91.5%、92.2%、92.3%）。这种差异的原因可能是我们的观测期晚于之前的工作（2019-2020 对比 2017），并且这些市场在受到审查后可能改变了它们的安全态势。每个应用商店的时间移除模式显示在表 8 中，表明所有应用商店都持续移除 PHA。需要注意的是，表 8 并未表明 Google Play 和华为市场不可信。相反，由于这些市场的流行及其庞大的用户基础，它们因此成为 PHA 制造商的法定目标。例如，Google Play 移除了 74K 个 PHA

3<https://www.statista.com/statistics/266210/number-of-available-applications-in-the-google-play-store/>

市场	平均持久性	恶意软件持久性	MUwS持久性
Google Play	77.64 D	78.72 D	77.11 D
Huawei Market	30.02 D	28.70 D	37.61 D
Xiaomi Market	29.93 D	27.40 D	37.27 D
Samsung Market	52.56 D	48.44 D	81.01 D
Bazaar Market	65.76 D	65.73 D	65.43 D
Oppo Market	28.29 D	26.32 D	36.47 D

谷歌 Play 移除了 74K 个 PHAs 表 9: 前六个 Android 市场中 PHA 的市场持续情况总结。

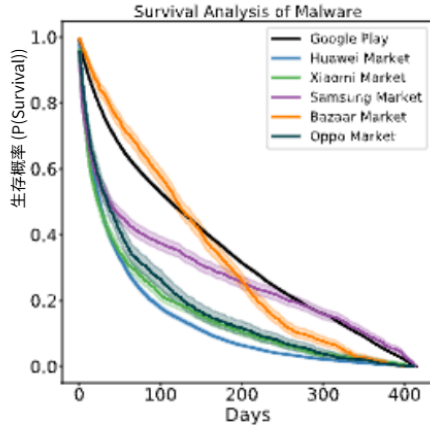


图 4: 六个市场中 PHAs、恶意软件和 MUwS 的生存分析。

在我们的观察期间, 其数量分别是小米市场和三星市场的 8 倍和 9 倍。

这与 Lindorfer 等人[22]的研究结果一致。

6.3 不同类型 PHAs 的市场持续存在

一个重要但尚未解决的问题是如何长时间 PHAs 能在不同市场存在才被移除, 因为它们存在的时间越长, 可能被感染的设备就越多。为了回答这个问题, 我们遵循第 3.3 节中概述的方法来测量这些 Android 市场的 PHA 在市场中的持久性。我们的研究结果总结在表 9 中。我们观察到, PHAs 平均可以在 Google Play 存在 77.64 天, 在其他市场至少存在 24 天。这为恶意行为者利用移动设备提供了很大的机会, 将用户及其数据置于风险之中。为了进一步研究我们的发现对平均 PHA 在市场中持久性的意义, 我们使用 Kaplan-Meier 估计[14]。回想我们的方法允许我们包含被删失的数据 (见第 3 节), 因此我们的估计是不偏的也不低估。六个市场中 PHAs 的生存分布如图 4 所示。从视觉上明显可以看出, 在时间轴上的任何一点, 我们都可以看到 Google Play 中 PHAs 的生存概率都高于其他市场 (除 Bazaar Market)。我们进一步进行了配对

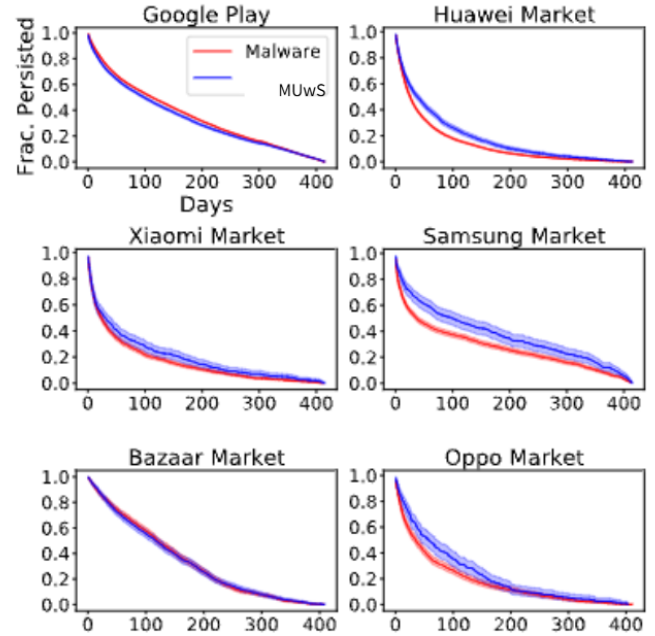
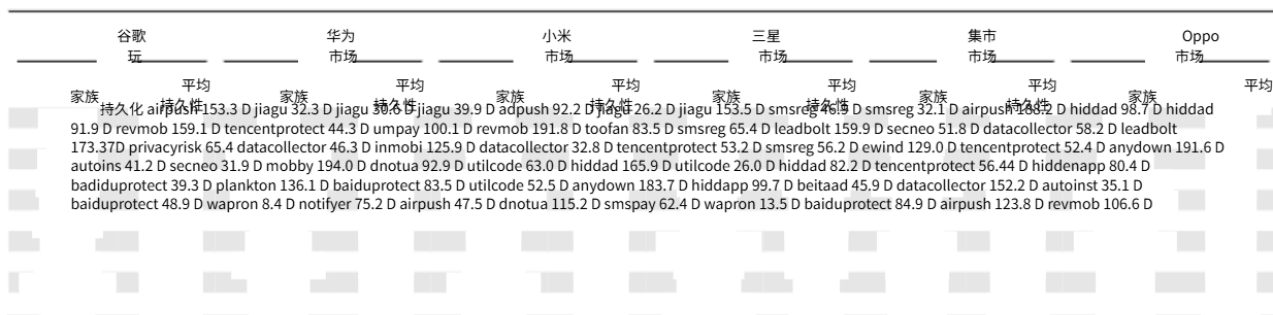


图 5: 六个市场中的恶意软件和 MUwS 持久性生存分析。

采用 wise Peto-Prentice 检验来比较谷歌和其他市场的 PHAs 生存分布, 以证明谷歌 Play 中的 PHAs 比其他市场中的 PHAs 持续时间更长。自由度是组数减一, 因此在我们的测试中始终为 1。 χ^2 检验表明这些差异具有统计学意义, 因为检验统计量值显著大于 3.841 (来自标准 χ^2 分布表), 并且 p 值都小于 0.005。这些结果进一步验证了我们的观察。我们的观察与 Lindorfer 等人[22]的观点相悖。我们假设有两个因素可能导致我们的结果。首先, 安卓在全球智能手机市场占 87%, 不可避免地已成为移动恶意软件的法定目标。反过来, 一些 PHAs 可能尽管谷歌加强了安卓的安全性和应用审核, 最终仍出现在谷歌 Play 商店。其次, 谷歌 Play 可能对 PHAs 有不同的政策 (例如, 它可能为这些 PHAs 提供更长的宽限期以移除有害库/代码)。然而, 在我们的观察期间, Google Play 移除了 74K 个 PHAs, 这远远超过了其他市场移除的数量。

然后我们使用 AVClass [29] 来区分恶意软件和 MUwS 之间的 PHAs, 调查这两种类型的 PHAs 是否在市场上存在任何持续性差异。移动 MUwS 在 Google Play (平均 77.11 天) 和 Bazaar 市场 (平均 65.43 天) 的持续性期略短, 而在其他四个市场中, MUwS 的持续性比恶意软件更长。特别是, 在三星市场中, 移动 MUwS 的市场持续性期几乎是恶意软件的两倍长。



移动 MUwS 在三星市场的市场持续期几乎是恶意软件的两倍长 表 10: 前 10 个家族 (按 SHA2 数量排名) 在 6 个顶级安卓市场的市场持续期总结。如果某个家族的市场持续期低于平均水平 (见表 9), 则其家族名称会加粗显示。

(平均 81.01 天) 和 Oppo 市场 (平均 48.44 天)。这表明不同市场在审核 PHAs 时可能采取不同的政策, 并可能优先考虑某些类型的威胁而忽视其他类型的威胁。为了进一步验证这两种类型的 PHAs 在市场上的持久性差异, 我们再次使用 Kaplan-Meier 估计。图 5 展示了六种市场中的移动恶意软件和 MUwS 的生存曲线。我们进一步进行 Peto-Prentice 检验, 以比较每种市场内恶意软件和 MUwS 的生存分布。 χ 检验表明这些差异具有统计学意义, 因为检验统计量的值显著大于 3.841 (来自标准 χ 分布表), 并且 p 值均小于 0.005。唯一的例外是 Bazaar 市场, 其中检验统计量不显著。因此, 我们不能得出 Bazaar 市场在审核 PHAs 时是否采取不同政策的结论。

在第五节中, 我们展示了感染设备总数与 SHA2 数量之间存在相关性。基于这一发现, 我们进一步研究了拥有大量 PHA 的 PHA 家族是否能在市场上存在更长时间。我们的假设是, 这些大型家族可能因为应用审核系统需要机器和人工双重检查, 所以在市场上存在时间更长。我们在前六个最大市场中的前十大家族的研究结果如表 10 所示。我们观察到, 前六个市场中的大部分大型家族的存在时间都长于平均存在时间 (见表 9)。例如, Google Play 中所有前十大家族的市场存在时间至少为 115 天, 比平均 77.64 天的存在时间长 38 天。这些结果表明, 需要更全面的应用审核措施。

6.4 PHA 市场内演化

在之前的章节中, 我们展示了 PHAs 可以在市场中持续数周。在本节中, 我们旨在进一步了解 PHA 家族如何在市场中演变。例如, PHA 制作者可能会主动切换广告库以应对市场政策变化或从广告中获得更好的激励, 或者他们可能会修改其恶意代码以规避市场应用审核系统, 等等。请注意, 每个应用在特定市场中都有一个唯一的包名, 通过关联属于



图 6: PHA 在市场上的演变示例 (com.sinyee.babybus.season)。

市场 #	应用 #SHA2s	大约 SHA2s 根据 PHA	平均市场内持久性	平均进化差距
Google Play	1,349 7,883 ~6 250.1 D 66.5 D			
华为市场	320 1779 ~5 276.6 D 116.2 D			
小米市场	89 443 ~5 247.8 D 98.8 D			
三星市场	70 383 ~5 238.9 D 86.8 D			
市集市场	40 129 ~3 213.9 D 120.5 D			
Oppo 市场	43 234 ~5 227.0 D 98.4 D			

表 11: PHA 在市场上的演变特征。

通过将某个包名与其 VT 报告的 AVClass 结果关联起来, 我们可以追踪和测量某个应用是否随时间演变 (即, 某个特定 PHA 的 SHA2 是否在一段时间内属于至少两个 PHA 家族)。我们在图 6 中展示了一个例子, 其中来自 Google Play 的 com.sinyee.babybus.season 的 SHA2 在观察期间与三个不同的 PHA 家族 (即, inmobi、kyview [36] 和 domob) 相关联。它们的整体演变分布如图 7 所示。从图中可以看出, 在市场上表现出演变特征的多数 PHA 主要出现在 Google Play 和华为市场 (这两个市场分别有 1,340 和 320 个 PHA)。在这些 PHA 中, 有有限数量的 PHA

⁴ 请注意, inmobi 是谷歌的首选广告 SDK 合作伙伴。然而, 该库被多个移动安全产品标记为 MUwS, 并且过去曾泄露过敏感用户数据。事实上, inMobi 在 2016 年被 FTC 指控违反 COPPA。因此, 尽管我们承认 MUwS 的定义因平台而异, 我们在这篇论文中将 inmobi 标记为 PHA。

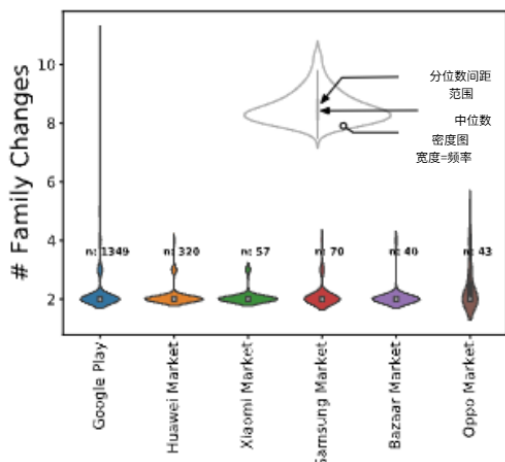


图 7: 紫色图示总结 PHA 市场内演变情况。中间的白点是中位数，中心的粗黑条表示四分位距，轮廓表示数据的分布形状。

其余市场也表现出市场内演变。例如，我们在三星市场识别出 10K 个 PHA（见表 8），但其中只有 70 个表现出市场内演变。平均而言，这些 PHA 随时间属于两个 PHA 家族。表现出市场内演变的这些 PHA 的其他特征总结在表 11 中。总体而言，这些表现出市场内演变的 PHA 在排名前 6 的市场中表现出更长的市场内持久性（即超过 200 天）。例如，表现出市场内演变的 PHA 在 Google Play 上的持久时间为 250.1 天，而平均时间为 77.6 天（见表 9）。在 Google Play 中 PHA 家族转换的平均间隔为 66.5 天，这比其他市场更频繁。我们认为 Google Play 的间隔较短部分是由于 Google Play 实施的严格的应用审核系统和安全政策。因此，恶意行为者必须主动应对来自 Google 的审查。

7 PHA 迁移

当他们的 PHA 从某个市场被移除时，恶意行为者可能会迁移到其他市场以继续他们的活动。在本节中，我们研究 PHA 如何在市场之间迁移。

7.1 PHA 跨市场迁移

一个重要的问题是，恶意行为者是否主动将 PHA 在不同市场之间迁移以感染更多设备，还是仅在 PHA 被某个市场移除后才会这样做。例如，恶意行为者可能在 Google Play 将其下架后将 PHA 迁移到其他市场。或者，恶意行为者可能将 PHA 从其他市场迁移到 Google Play，以在短时间内从其庞大的终端用户中获利。我们在图 8 中展示了一个例子。我们有一个重打包的应用程序 com.avatar.star，其 SHA2 为 79e6...，它最初...

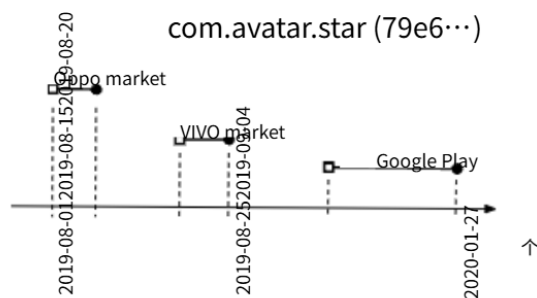


图 8: PHA 跨市场迁移示例 (com.avatar.star)。

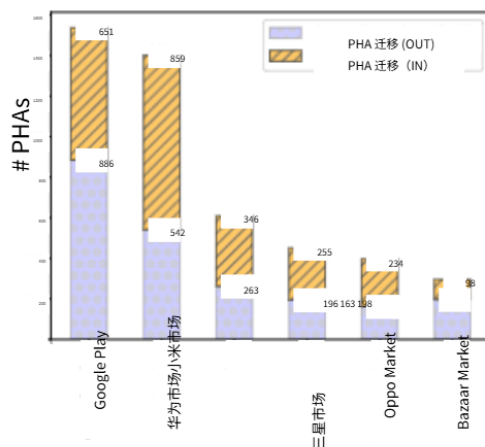


图 9: PHA 跨市场迁移。市场按跨市场活动的总数排名。

最终于 2019 年 8 月 1 日至 2019 年 8 月 15 日在 Oppo Market 出现（3 台设备感染），然后迁移至 VIVO Market（2019 年 8 月 20 日至 2019 年 8 月 25 日，2 台设备感染），最后在 Google Play（2019 年 9 月 4 日至 2020 年 1 月 27 日，215 台设备感染）安顿下来。回想一下，我们可以跨市场和时间追踪 SHA2，以识别连续的非重叠时间间隔。为了量化上述现象，我们使用应用程序包名及其关联的 SHA2 作为种子（类似于先前的工作[22]），并使用第 3 节中详细描述的方法来测量 PHA 跨市场迁移。总共，我们观察到 3,533 个 PHA 表现出跨市场迁移。前六个市场的结果总结在图 9 中。我们看到 Google Play 表现出最多的跨市场迁移活动，有 1,404 次 PHA 迁移，同时 Google Play 也表现出最多的向外 PHA 迁移，有 886 次向外迁移活动。

我们接下来研究移动恶意软件和 MUwS 在不同的市场上是否表现出不同的迁移活动。我们再次使用 AVClass [29]来识别移动恶意软件和

市场	总 PHA 迁移 (in)	# 恶意软件 # MUwS	平均持久性	# 感染设备 (upstream)	# 设备感染 (current)
谷歌 Play	651 447 204 57.9 D	964 3,063			
华为市场	859 747 112 63.5 D	1,039 1,543			
小米市场	346 292 54 10.44 D	4,065 471			
三星市场	255 218 37 23.66 D	1,599 394			
Oppo 市场	234 186 58 10.3 D	3,364 296			
市集	107 63 44 63.84 D	121 284			

表 12：市场对 PHA 迁移的响应。

我们再次使用 AVClass [29] 来识别从包名迁移的移动恶意软件和 MUwS。Lindorfer 等人 [22] 发现了初步证据表明恶意应用会从一个市场跳到另一个市场，可能是为了生存。例如，作者们识别了 131 个迁移到替代市场的应用，但没有进一步分析这些应用在迁移后能存活多久。为了填补这一空白，我们测量了具体迁移到市场中的 PHA，以了解它们在市场中的持久性。结果总结在表 12 中。与 MUwS 相比，在所有前 6 个市场中，更多的移动恶意软件迁移到市场。我们的假设是，MUwS 的生态系统通常利用广告库，并且更能适应市场下架，而移动恶意软件背后的恶棍使用更复杂的方法（例如，代码混淆、环境感知等），因此在不同的市场重用相同的 PHA 以最大化受害者数量是更可取的。为验证我们的假设，我们测量了这些迁移至市场的 PHAs 的设备流行率，并将这些流行率与它们直接上游市场迁移来源的流行率进行比较。我们的结果总结在表 12 中。从表中可以看出，迁移至用户基数庞大的 Google Play 和华为市场的 PHAs 能够感染至少比直接上游市场多 50% 的设备。然而，迁移至其他用户基数较小的市场的 PHAs 并未感染更多设备。尽管如此，这些 PHAs 在这些市场的平均寿命相对较短（与平均持久时间相比，见表 9，第 6 节），除了华为市场。我们的假设是，这部分是由于这些 PHAs 在上游市场已被检测到，因此下游市场可以获取签名来检测它们。与此同时，华为市场的例外情况表明，市场必须负责并严格审核提交的应用程序。我们的研究仅测量了 PHA 在市场上的最低持久性，因为有可能 PHA 仍然存在于市场上，但我们的数据集未能反映其存在。如果我们的数据集通过 Lindorfer 等人提出的方法 [22] 进行增强，这个问题可以得到解决。我们将此类任务作为我们未来工作的部分。

服务 #PHAs #恶意软件 #MUwS	开发感染	平均持久性
com.sec.android.easyMover (Samsung)	14,038 10,960 3,078 35,557 93.38 D	
com.samsung.android.scloud (Samsung)	5,088 3,835 1,253 8589 56.41 D	
com.hicloud.android.clone (Huawei)	3,653 2,953 700 3,079 32.53 D	
com.oneplus.backupstore (Oneplus)	1,072 794 278 1,361 22.69 D	
com.coloros.backupstore (Oppo)	972 695 277 1,267 21.98 D	
com.miui.cloudbackup (Xiaomi)	1,243 928 315 1,235 33.23 D	

表 13：从数据备份/克隆服务迁移 PHA 这些服务是根据设备普及率进行排名的。

7.2 PHA 持久化：通过备份/克隆服务迁移后

安卓手机通常为用户提供备份功能，允许他们在购买新设备时恢复他们的应用和配置。这种机制使用户能够在新设备上快速恢复他们的数据（例如，联系人、设置、应用），而无需手动重新安装。然而，这些服务可能会无意中现有的 PHA 迁移到新设备上，从而危及新手机的安全和隐私，即使这些 PHA 可能已被市场移除，因此用户可能无法再手动安装它们。Kotzias 等人 [16] 表明，备份恢复是一种非预期的、不受欢迎的应用程序分发渠道，负责 4.8% 的不受欢迎的安装。沿着这个方向，我们进一步研究了 PHA 在通过备份/克隆服务迁移后可以持续多长时间。回想一下，移动安全产品捕获了应用程序的安装程序包名（见第 2 节）。这使我们能够识别我们数据集中由备份/克隆服务安装的应用程序。为此，我们首先识别数据集的前六大数据备份/克隆服务，了解有多少恶意软件（PHAs）从这些服务迁移，并因此了解这些恶意软件可能在这些设备上持续多长时间。为了准确识别数据备份/克隆服务，我们首先移除所有已知的市场安装包，然后根据设备普及率对剩余的安装包进行排名。随后，我们在 Google Play 和网络上调查这些应用程序，以了解安装包的功能。

我们的研究结果如表 13 所示。总体而言，我们观察到相当数量的恶意软件没有被用户移除，因此从旧手机和备份中迁移。例如，在我们的数据集中，14K 恶意软件迁移到了 35.5K 部新的三星设备。同时，有趣的是我们发现通过备份/克隆服务迁移的移动恶意软件数量是 MUwS 的三倍。此外，这些恶意软件的持续期比平均 20.2 天的持续期（见表 4）更长。例如，通过三星智能切换移动应用程序（com.sec.android.easyMover）迁移的恶意软件在新设备上平均持续 93 天。



图 10：案例研究：模拟应用家族

8 欺骗应用案例研究

在本节中，我们以欺骗应用安卓恶意软件家族为例进行案例研究，以证明我们的方法可以测量恶意活动如何传播、持续存在以及最终被市场移除。欺骗应用是一个恶意应用家族，它伪装成流行的合法应用，通过使用与安全应用、银行应用等相似的包名和图标。欺骗应用家族中的某些应用可能会从事发送/接收高级短信消息和下载其他应用等恶意活动[17,40]。

图 10 展示了假应用从全球和本地视角的流行率和设备感染率（在我们的案例研究中，我们使用了 Google Play 市场）。在我们的观察期间，从全球范围来看，假应用每月大约有 3K 个活跃的 SHA2，感染 13K 台设备（见图 10a 和 10b）。从本地视角来看，假应用在 Google Play 市场中有大约 210 个活跃的 SHA2 持续存在，每月感染 3K 台设备（见图 10c 和 10d）。这相当有趣，因为 Google Play 市场中的假应用家族的应用占该家族每月活跃的 SHA2 的 7%，但却占全球设备感染的约 25%。我们的研究表明，Google Play 市场在我们的观察期间加大了移除力度。该市场在 2019 年 1 月有 322 个活跃的假应用 SHA2，并在 2020 年 2 月减少到 106 个活跃的 SHA2，减少了两倍（图 10c）。这反过来导致设备感染率减少了四倍（图 10d）。然而，假应用家族平均在 Google Play 市场存在 86.6 天后才被移除。这比该市场的平均 PHA 持续期长约 9 天（见第 6.2 节表 9）。同时，从 Google Play 安装的 fakeapp 应用持续 29 天。41 天在设备上，这是比平均 15 天长 14 天

17 天设备上的假应用持久期。我们认为这是由于 Google Play 是事实上的安卓应用可信来源，因此最终用户可能会更长时间地保留来自 Google Play 的 PHAs。此外，假应用伪装成有用应用的事实可能掩盖了这些应用具有恶意行为，并诱使用户在收到移动安全产品的警告后仍将它们保留在设备上更长时间。我们的案例研究突出了 Google Play 市场在对抗 PHAs 方面的重要性，并展示了我们的测量方法可以实现深入分析。

9 局限性与讨论

偏见。尽管本文展示了迄今为止最大规模的设备端 Android PHA 测量结果，但我们的数据集偏向于单一移动安全产品的用户，因此仍然存在一些偏见。例如，我们的设备群体偏向于美国和欧洲国家。美国和欧洲的终端用户可能倾向于更长时间地保留这款移动安全应用，因此这些设备更有可能符合我们的数据选择标准（见第 3 节）。同时，我们无法观察到未使用移动安全产品的用户以及未选择参与此次数据收集方案的用户的行为。此外，我们无法观察到受 Google Play Protect 保护的设备发生的事件。尽管如此，我们相信我们的数据集能够代表全球移动用户，并尽力通过使用百分比来查看各国的感染率等方式来最小化这种偏见。在分析应用代表性方面，由于安卓生态系统高度碎片化且存在众多替代市场，确定本研究覆盖范围具有挑战性，因为无法确定所有安卓应用的总数。尽管如此，通过分析 8.8M 个独特应用，本研究覆盖了迄今为止最大规模的应用集合之一，并与学术界收集的最大数据集保持一致[1]。

数据限制。需要注意的是，PHA 检测数据是被动收集的。也就是说，当安全产品检测到符合预定义特征（包括其行为、通信和政策违规）的潜在有害应用时，会记录一次 PHA 检测事件。其他安全产品（例如应用商店链接黑名单、基于云的应用声誉系统）预先阻止的任何移动 PHA 无法被观察到。此外，设备上任何不符合预定义特征的 PHA 也无法被观察到。由于移动安全数据是被动收集的，因此推断 PHA 在市场上的最后出现时间实际上很困难。我们的推断依赖于这样的逻辑：如果我们从 11.7M 设备在最后一次观察到该 PHA 之后生成的数十亿事件中都没有观察到该 PHA，我们就认为该 PHA 已被市场移除。该 PHA 仍可能留在该市场，而我们的数据集只是未能捕捉到它的存在。

有可能这个 PHA 仍然留在那个市场, 而我们的数据集只是没有捕捉到它的存在 (即, 这个 PHA 在其最后观察时间之前没有被观察到)。因此, 我们

Android API 使移动安全产品能够识别 PHA 的安装器包名。通过与市场官方包名进行关联, 我们可以确定 PHA 在特定时间是否来自某个市场。然而, 恶意用户或最终用户可以通过 ADB 安装应用程序, 并冒充市场官方包名。在这种情况下, 移动安全产品可能会错误地将 PHA 归因于某个市场。为了降低这种风险, 我们的研究仅选择在至少两台设备上观察到的 PHA。我们认为, 由冒充的官方市场包名所导致的此类误报在统计上是可忽略的。此外, 如果应用程序在我们观察期开始前已安装, 我们无法获取其市场信息。如果已安装的应用程序随后被更新, 我们的系统会将更新软件视为安装者, 而不是应用程序最初来自的市场。因此, 我们排除了那些我们无法确信归因于特定市场的 PHA。尽管如此, 这仍然使我们能够覆盖数据集中 66% 的设备以及所有 PHA 安装的 22%。

对移动安全研究的影响。我们的研究表明, 一旦安装或批准, 许多 PHAs 可以在设备和应用市场上持续存在多日。我们希望我们的研究能够启发更好的通知系统, 在检测到 PHAs 时促使最终用户将其移除, 并且理想情况下, 设计一个能够说服用户从一开始就不安装 PHAs 的预防系统。

对安卓市场的启示。我们的研究表明, 恶意软件 (PHAs) 可以在市场上持续存在至少 24 天。与此同时, 虽然我们承认安卓市场的努力, 但并非所有恶意软件都被它们移除 (例如, Google Play 每月移除 5.28K 个恶意软件, 总共移除了 81K 个恶意软件中的 74K 个)。我们希望我们的研究结果能够促使安卓市场加强其应用审核系统, 及时移除恶意软件, 以最大限度地减少其在市场上的持续存在。此外, 尽管有 Google Play 的透明度报告, 但我们希望市场能够更加透明, 并披露与恶意软件移除相关的性能数据 (例如, 每月移除的恶意软件数量、移除恶意软件的平均时间等)。我们的研究表明, 恶意软件可能会随时间演变, 以在市场上生存更长时间并能够触达更多受害者。我们希望我们的研究结果能够鼓励应用市场让最终用户了解应用先前版本可能存在的安全和隐私问题。例如, 在 Google Play 上的流行应用 com.intsig.camscanner 的某些版本因集成了 AdHub 的第三方 SDK 而受到了木马投递器 necro 的影响。在移除第三方库后, 该应用仍然留在 Google Play 上

关于此类应用相关的安全和隐私事件的简史, 将能为终端用户在将来安装这些应用到他们的设备上提供明智的决策。

10 相关工作

在移动安全和隐私方面有大量的研究。在本节中, 我们特别回顾了关于恶意软件特征化和移动应用生态系统的先前测量研究。我们建议读者参考[6, 8, 19, 24, 32, 37]以获取关于保护 Android 设备概述和调查。

移动 PHA 特征分析。安全研究界多年来一直积极研究不断变化的移动 PHA 特性[6, 8, 19, 24, 32, 37]。以往的努力主要集中在分析应用程序, 并从各个方面系统地对其进行特征分析。从宏观层面来看, 这些研究主要关注安装方法[40]、规避机制[7]、重打包机制[22,31,39]、恶意载荷[40]、行为[21, 38]、盈利模式[9]等。近年来, Faruki 等人[7]总结了 Android 安全问题、恶意软件增长 (2010-13 年期间)、其渗透方式、隐身技术以及一些流行缓解方案的优势和劣势。Mirzaei 等人[23]介绍了 Andrensemble 系统, 该系统通过利用 API 集合来对 Android 恶意软件家族进行特征分析。这些努力共同揭示了安卓恶意软件在现实中的运作方式、移动恶意软件的主要动机、一些流行缓解解决方案的弱点等。然而, 它们没有讨论 PHA 持久化在移动设备和市场所带来的潜在威胁, 因为这些努力集中于应用分析, 且对真实设备普及情况提供了不太全面的观点。

关于 Android 权限系统的测量研究。Android 权限系统在之前的文献中已被广泛覆盖[2, 3, 8, 24]。我们仅在本论文中回顾与我们研究相关的工作。Felt 等人[8]构建了 Stowaway 系统来检测可能导致隐私侵犯的过度授权应用。Felt 等人[10]后来表明, 当前的 Android 权限警告并不能帮助大多数用户做出正确的安全决策。Sarma 等人[27]讨论了 Android 权限系统带来的风险, 并概述了 13 项可能严重侵犯用户隐私的权限。Qu 等人[25]设计了 AutoCog 来测量 Android 应用中描述到权限的保真度, 并帮助最终用户在授予权限时理解其安全性和隐私影响。

关于移动 PHA 的测量研究。从设备相应地, Shen 等人[30]对 614 万台安卓设备进行了详细的定量分析, 比较了在广泛特征 (包括 PHA 安装和网络行为) 方面 rooted 和 non-rooted 安卓设备。Suarez-Tangil 等人[31]对 2010 年至 2017 年间跨越的 128 万款重打包应用进行了系统研究, 以了解安卓恶意软件是如何随时间演变的。

最近, Gamba 等人[11]收集了来自 200 多家供应商的 82K 预装应用(总共 424K 个文件)的安卓设备,并进行了测量研究,以了解利益相关者主要如何围绕广告和数据驱动服务建立他们的关系。从应用市场角度来看, Lindorfer 等人[22]提出了 AndRadar 系统,使用一组包名作为种子,在一系列替代应用市场中发现恶意安卓应用的多个实例。王等人[35]利用从 16 个中国应用市场和 Google Play 下载的 600 万个安卓应用,提供了一项大规模的比较研究,以了解与应用(包括 PHAs)相关的各个方面和动态,以及它们的行为和开发者。这些努力共同揭示了 PHA 随时间演变整体图景。与这些先前的努力不同,我们的研究关注 PHA 在移动设备和市场中的潜在威胁,因为这些努力集中在应用分析上,并提供了对真实设备流行情况的全面视图。

台式机 PUP PPI 生态系统研究。另一个松散地相关的研究线涉及在 PC 环境下测量 PUP PPI 生态系统。Caballero 等人[4]在野外对黑市按安装付费服务进行了首次大规模测量。Kotzias 等人[15]利用文件下放图构建了发布者图,并识别了生态系统中的特定角色,进而揭示了 PUP 流行与 PUP 分销商之间的关系。Thomas 等人[33]对台式机上的恶意软件进行了类似的研究。

与近距离工作的比较。最接近的工作是一项重新由 Kotzias 等人进行的移动设备非期望应用分发研究[16]。他们的研究重点在于理解安装者和子应用之间的“谁安装谁”关系,并揭示主要的非期望应用分发渠道。与 Kotzias 等人[16]的研究发现相似,我们的研究也表明 Google Play 仍然是 PHA 的主要分发渠道,同时也拥有对 PHA 的最佳防御措施(例如,移除大部分 PHA)。Kotzias 等人[16]还识别了其他许多分发渠道,如臃肿软件、浏览器、即时通讯工具等。我们的研究不涵盖这些分发渠道,因为我们专注于 PHA 的时间行为。具体来说,通过利用跨越 1100 万设备的更长时间段的 PHA 安装事件观察期,我们的研究提供了对 Android PHA 的大规模时间测量研究,以理解它们在设备上的持续性和在市场上的持久性特征,以及在被下架后的市场间迁移情况。总之, Kotzias 等人[16]关注 PHA 的来源,而我们的研究则探讨 Android 上 PHA 安装的时间动态。

11 结论

我们展示了迄今为止最大规模的在设备上对野外安装的 Android PHAs 的研究

我们的结果表明, Android 上的 PHAs 是一个普遍存在的问题, 恶意应用可以在设备和市场上长期存在。我们的结果表明, 目前针对 Android 上恶意应用的措施并不像人们普遍认为的那样有效, 并且安全社区需要在这个领域进行更多研究。

致谢

我们感谢我们的导师 Yousra Aafer 和匿名审稿人在修改过程中的有益指导。这项工作得到了国家自然科学基金通过 CNS-2127232 资助。

参考文献

[1] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, 和 Yves Le Traon. Androzoo: 为研究社区收集数

[2] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, 和 David Lie. Pscout: 分析安卓权限规范. 在 ACM CCS, 2010.

[3] David Barrera, H Güneş Kayacik, Paul C Van Oorschot, 和 Anil Somayaji. 基于权限的安全模型的实证分析方法及其在安卓上的应用. 在 ACM CCS, 2010.

[4] Juan Caballero, Chris Grier, Christian Kreibich, 和 Vern Paxson. 测量按安装付费: 恶意软件分发的商品化. 在 USENIX Security, 2011.

[5] Rahul Chatterjee, Periwinkle Doerfler, Hadas Orgad, Sam Havron, Jackeline Palmer, Diana Freed, Karen Levy, Nicola Dell, Damon McCoy, 和 Thomas Ristenpart. 亲密伴侣暴力中使用的间谍软件.

在 IEEE S&P, 2018.

[6] Zheran Fang, Weili Han, and Yingjiu Li. 基于权限的安卓安全: 问题与对策. computers & security, 43, 2014.

[7] Parvez Faruki, Ammar Bharmal, Vijay Laxmi, Vijay Ganmoor, Manoj Singh Gaur, Mauro Conti, and Muttukrishnan Rajarajan. Android 安全: 问题、恶意软件渗透及防御的调查. IEEE 通信调查与教程, 17(2), 2014.

[8] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. Android 权限解密. 在 ACM CCS, 2011.

[9] Adrienne Porter Felt, Matthew Finifter, Erika Chin, Steve Hanna, and David Wagner. 野外移动恶意软件调查. 在 SPSM, 2011.

- [10] Adrienne Porter Felt, Elizabeth Ha, Serge Egelman, Ariel Haney, Erika Chin, 和 David Wagner. Android 权限: 用户注意力、理解力和行为。在 CCS, 2012.
- [11] Julien Gamba, Mohammed Rashed, Abbas Razaghpanah, Juan Tapiador, 和 Narseo Vallina-Rodriguez. 预装安卓软件的分析。在 IEEE S&P, 2020.
- [12] Google. Android 安全与隐私 2018 年度回顾。2019.
- [13] Médéric Hurier, Guillermo Suarez-Tangil, Omar Darwish, Tegawendé F Bissyandé, Yves Le Traon, Jacques Klein 和 Lorenzo Cavallaro. Euphony: 和谐统一嘈杂的反病毒供应商标签用于安卓恶意软件。在 CCS, 2017.
- [14] David G Kleinbaum 和 Mitchel Klein. Survival analysis. Springer, 2010.
- [15] 普拉顿·科齐亚斯, 华拉·比尔执和胡安·卡瓦列罗. 通过付费安装服务监测 PUP 流行率和 PUP 分布。在 USENIX Security, 2016.
- [16] 普拉顿·科齐亚斯、胡安·卡瓦列罗和莱拉·比尔吉. 如何那东西进我手机了吗? 安卓设备上的非自愿应用分发。IEEE S&P, 2021.
- [17] 苏蒙凯维, 英久李, 罗伯特·邓, 杰森洪. 在移动应用市场检测伪装应用。在 ICISC, 2014.
- [18] Charles Lever, Manos Antonakakis, Bradley Reaves, Patrick Traynor, 和 Wenke Lee. 问题的核心: 分析蜂窝运营商中的恶意流量。在 NDSS, 2013.
- [19] Li Li, Tegawendé F Bissyandé, 和 Jacques Klein. 重新检测重新打包的安卓应用的研究: 文献综述和基准测试。IEEE Transactions on Software Engineering, 2021.
- [20] 李丽, 道原李, Tegawendé F Bissyandé, 雅克克莱因、伊夫·勒·特隆、大卫·洛和洛伦佐·卡瓦拉罗. 理解安卓应用搭便车: 恶意代码嫁接的系统研究。IEEE 信息取证与安全汇刊 (TIFS), 2017.
- [21] 马丁娜·林多弗尔, 马蒂亚斯·纽格施万特纳, 卢卡斯 Weichselbaum、Yanick Fratantonio、Victor Van Der Veen 和 Christian Platzer. AndrubiS-1,000,000 个应用之后: 对当前安卓恶意行为的一种调查。在 RAID, 2020.
- [22] Martina Lindorfer, Stamatis Volanis, Alessandro Sisto, Matthias Neugschwandtner, Elias Athanasopoulos, Federico Maggi, Christian Platzer, Stefano Zanero 和 Sotiris Ioannidis. Andradar: 快速发现替代市场中的安卓应用。在 DIMVA, 2014.
- [23] Omid Mirzaei, Guillermo Suarez-Tangil, Jose M de Fuentes, Juan Tapiador, 和 Gianluca Stringhini. Andrensemble: 利用 API 集成来表征安卓恶意软件家族。在 ASIACCS, 2018.
- [24] Mohammad Nauman, Sohail Khan, 和 Xinwen Zhang. Apex: 扩展安卓权限模型和执行机制, 通过用户定义的运行时约束。在 ASIACCS, 2010.
- [25] 郑阳区, Vaibhav Rastogi, 张新怡, Yan 陈, 朱天添, 和 陈中. Autocog: 测量安卓应用描述到权限的保真度。在 ACM CCS, 2014.
- [26] Abbas Razaghpanah, Rishab Nithyanand, Narseo Vallina-Rodriguez, Srikanth Sundaresan, Mark Allman, Christian Kreibich, 和 Phillipa Gill. 应用, 追踪器, 隐私, 和监管者: 对移动追踪生态系统的安全研究。在 USENIX, 2016.
- [27] Bhaskar Pratim Sarma, Ninghui Li, Chris Gates, Rahul Potharaju, Cristina Nita-Rotaru, 和 Ian Molloy. Android 权限: 结合风险与收益的视角。在 SACMAT, 2016.
- [28] 安吉拉·萨塞. 将人们吓唬和欺负到安全不会起作用。IEEE Security & Privacy, 13(3):80–83, 2015.
- [29] 马科斯·塞巴斯蒂安, 理查德·里韦拉, 普拉顿·科齐亚斯和胡安·卡瓦列罗. Avclass: 一种用于大规模恶意软件标注的工具。在 RAID 会议, 2016.
- [30] 云神, 内森·埃文斯和阿兹丁·巴内穆尔. 在-通过行为分析查看已 Root 和未 Root 的安卓移动设备。在 SAC, 2016.
- [31] Guillermo Suarez-Tangil 和 Gianluca Stringhini. 八年在安卓恶意软件生态系统中的骑行测量。IEEE Transactions on Dependable and Secure Computing, 2020.
- [32] Darell JJ Tan, Tong-Wei Chua, Vrizlynn LL Thing, 等. 保护安卓: 一项调查、分类和挑战。ACM 计算机调查 (CSUR), 47(4), 2015.
- [33] Kurt Thomas, Juan A Elices Crespo, Ryan Rasti, Jean-Michel Picod, Cait Phillips, Marc-André Decoste, Chris Sharp, Fabio Tirelo, Ali Tofigh, Marc-Antoine Courteau, 等. 调查商业按安装付费和不良软件的传播。在 USENIX 安全研讨会, 2016.

- [34] 韩宇王, 李浩, 李丽, 郭耀, 徐国爱。为什么安卓应用会被从谷歌应用商店移除? 一项大规模实证研究。在 MSR, 2018。
- [35] 韩宇王, 刘哲, 梁静月, Narseo Vallina-Rodriguez, 姚国, 李丽, 胡安·塔皮亚多, 曹景存, 和徐国爱。超越谷歌 Play: 中文安卓应用市场的规模化比较研究。在 2018 年互联网测量会议论文集, 2018 年。
- [36] 冯国伟, 李玉平, Sankardas Roy, Ou Xinming, 和吴周。当前安卓恶意软件的深度真实情况分析。在 DIMVA, 2017。
- [37] 孟旭, 宋成宇, 季阳, 石明伟, 卢康杰, 郑聪, 段瑞安, 姜永镇, 李丙英, 钱晨雄, 等。迈向构建安全的安卓生态系统: 现有研究综述。ACM 计算机调查 (CSUR), 49(2), 2016。
- [38] 曹阳, 徐昭岩, 顾国飞, Vinod Yegneswaran 和 Phillip Porras. Droidminer: 自动挖掘和表征安卓应用中的细粒度恶意行为。在 EC2SPICE, 2014。
- [39] 吴周, 姚金, 江旭仙, 和宁鹏。在第三方安卓应用市场中检测重新打包的智能手机应用。在 CODASPY, 2012。
- [40] 周亚金和江旭仙。解构安卓恶意ware: 特性和演化。在 IEEE S&P, 2012。
- [41] 罗飞, 石建军, 杨立明, 秦博琴, 子怡, 张林海, 宋刚和王刚。测量和建模在线反恶意软件引擎的标签动态。在 USENIX 安全研讨会, 2020。